스토리지 백엔드

소개

Oasis Core는 다양한 스토리지 백엔드를 지원하여 서로 다른 성능 요구사항과 운영 환경에 맞춰 최적화된 저장소를 선택할 수 있습니다. 각 백엔드는 MKVS(Merklized Key-Value Store) 위에서 동작하며, 노드 데이터베이스 인터페이스를 구현합니다.

지원되는 백엔드

1. BadgerDB 백엔드

BadgerDB는 Go로 작성된 임베디드 키-값 데이터베이스로, LSM-tree 구조를 사용합니다.

특징

- **고성능**: SSD에 최적화된 LSM-tree 구조
- **ACID 트랜잭션**: 완전한 ACID 속성 지원
- **압축**: 자동 압축과 가비지 컬렉션
- **메모리 효율성**: 효율적인 메모리 사용

설정 옵션

```
type Config struct {
DB  string  // 데이터베이스 경로
Namespace common.Namespace // 네임스페이스
MaxCacheSize  int64  // 최대 캐시 크기
```

```
NoFsync bool // fsync 비활성화
MemoryOnly bool // 메모리 전용 모드
ReadOnly bool // 읽기 전용 모드
}
```

사용 예제

```
// BadgerDB 백엔드 생성
cfg := &api.Config{
DB: "/path/to/database",
Namespace: namespace,
MaxCacheSize: 64 * 1024 * 1024, // 64MB
NoFsync: false,
MemoryOnly: false,
ReadOnly: false,
}

nodeDB, err := badger.New(cfg)
if err != nil {
return err
}
defer nodeDB.Close()
```

2. PathBadger 백엔드

PathBadger는 BadgerDB를 기반으로 하지만 트리 경로를 키로 사용하는 최적화된 백엔드입니다.

특징

- **경로 기반 키**: 트리 경로를 직접 키로 사용
- **향상된 성능**: 노드 조회 성능 최적화
- **효율적 저장**: 중복 제거와 압축 최적화
- **멀티파트 복원**: 체크포인트 복원 최적화

주요 차이점

```
// 기존 BadgerDB: 해시 기반 키
nodeKeyFmt = keyFormat.New(0x00, &hash.Hash{})

// PathBadger: 경로 기반 키
finalizedNodeKeyFmt = keyFormat.New(0x01, uint8(0), []byte{})
pendingNodeKeyFmt = keyFormat.New(0x02, uint64(0), uint8(0), uint16(0), []byte{}
```

사용 예제

```
// PathBadger 백엔드 생성
cfg := &api.Config{
DB: "/path/to/pathbadger",
Namespace: namespace,
MaxCacheSize: 128 * 1024 * 1024, // 128MB
```

```
}


nodeDB, err := pathbadger.New(cfg)
if err != nil {
return err
}
defer nodeDB.Close()
```

백엔드 선택 가이드

성능 비교

| 백엔드 | 읽기 성능 | 쓰기 성능 | 저장 효율성 | 메모리 사용량 |

|--------|----------|----------|----------|----------|

| BadgerDB | 높음 | 높음 | 보통 | 보통 |

| PathBadger | 매우 높음 | 높음 | 높음 | 낮음 |

사용 시나리오

BadgerDB 권장 상황

- **일반적인 용도**: 대부분의 표준 사용 사례
- **안정성 우선**: 검증된 안정성이 필요한 경우
- **호환성**: 기존 시스템과의 호환성이 중요한 경우

PathBadger 권장 상황

- **고성능 요구**: 최대 성능이 필요한 경우
- **대용량 데이터**: 큰 상태 데이터를 처리하는 경우
- **체크포인트 복원**: 빈번한 체크포인트 복원이 필요한 경우

성능 최적화

1. 캐시 설정

메모리 캐시 크기 조정

```
cfg := &api.Config{
MaxCacheSize: 256 * 1024 * 1024, // 256MB
}

// 시스템 메모리의 10-20% 권장
totalMemory := getSystemMemory()
cfg.MaxCacheSize = totalMemory / 5
```

BadgerDB 특화 설정

```
opts := badger.DefaultOptions("/path/to/db")
opts.ValueLogFileSize = 64 << 20  // 64MB value log files
opts.MemTableSize = 64 << 20  // 64MB memtable
opts.NumMemtables = 5  // 5 memtables
opts.NumLevelZeroTables = 5  // Level 0 tables
```

```
opts.NumLevelZeroTablesStall = 10  // Stall threshold
```

2. 배치 작업 최적화

대량 삽입 최적화

```
// 배치 생성
batch, err := nodeDB.NewBatch(oldRoot, version, false)
if err != nil {
return err
}
defer batch.Reset()

// 여러 노드를 배치로 처리
for _, node := range nodes {
if err := batch.PutNode(node); err != nil {
return err
}
}

// 한 번에 커밋
if err := batch.Commit(newRoot); err != nil {
return err
}
```

멀티파트 복원 최적화

```
// 멀티파트 복원 시작
if err := nodeDB.StartMultipartInsert(version); err != nil {
return err
}
defer nodeDB.AbortMultipartInsert() // 실패 시 정리

// 청크별 배치 처리
for _, chunk := range chunks {
batch, err := nodeDB.NewBatch(root, version, true) // chunk=true
if err != nil {
return err
}
// 청크 데이터 처리
if err := processChunk(batch, chunk); err != nil {
batch.Reset()
return err
}
if err := batch.Commit(root); err != nil {
batch.Reset()
return err
}
```

```
}

// 복원 완료
if err := nodeDB.Finalize([]node.Root{root}); err != nil {
return err
}
```

3. 가비지 컬렉션 최적화

BadgerDB GC 설정

```
// GC 워커 시작 (자동으로 처리됨)
gc := cmnBadger.NewGCWorker(logger, db)
gc.Start()
defer gc.Stop()

// 수동 GC 트리거 (필요시)
err := db.RunValueLogGC(0.5) // 50% 임계값
```

버전 정리

```
// 오래된 버전 정리
earliestVersion := nodeDB.GetEarliestVersion()
latestVersion, _ := nodeDB.GetLatestVersion()
```

```
// 최근 100개 버전만 유지
if latestVersion-earliestVersion > 100 {
pruneVersion := latestVersion - 100
if err := nodeDB.Prune(pruneVersion); err != nil {
return err
}
}
```

백업과 복구

1. 데이터베이스 백업

전체 백업

```
func backupDatabase(nodeDB api.NodeDB, backupPath string) error {
// 데이터베이스 크기 확인
size, err := nodeDB.Size()
if err != nil {
return err
}
fmt.Printf("Backing up database (size: %d bytes)\n", size)
// 백업 수행 (구현체별로 다름)
switch db := nodeDB.(type) {
case *badger.BadgerNodeDB:
```

```
return backupBadgerDB(db, backupPath)
case *pathbadger.BadgerNodeDB:
return backupPathBadgerDB(db, backupPath)
default:
return fmt.Errorf("unsupported database type")
}
}
```

증분 백업

```
func incrementalBackup(nodeDB api.NodeDB, lastBackupVersion uint64) error {
latestVersion, exists := nodeDB.GetLatestVersion()
if !exists {
return fmt.Errorf("no versions available")
}
// 변경된 버전들만 백업
for version := lastBackupVersion + 1; version <= latestVersion; version++ {
roots, err := nodeDB.GetRootsForVersion(version)
if err != nil {
return err
}
if err := backupVersion(nodeDB, version, roots); err != nil {
return err
```

```
}
}
return nil
}
```

2. 데이터베이스 복구

체크포인트 기반 복구

```
func restoreFromCheckpoint(nodeDB api.NodeDB, checkpointPath string) error {
// 체크포인트 메타데이터 로드
metadata, err := loadCheckpointMetadata(checkpointPath)
if err != nil {
return err
}
// 멀티파트 복원 시작
if err := nodeDB.StartMultipartInsert(metadata.Root.Version); err != nil {
return err
}
defer nodeDB.AbortMultipartInsert()
// 청크별 복원
for i, chunkHash := range metadata.Chunks {
chunkData, err := loadChunk(checkpointPath, i)
if err != nil {
```

```
return err
}
if err := restoreChunk(nodeDB, metadata.Root, chunkData); err != nil {
return err
}
}
// 복원 완료
return nodeDB.Finalize([]node.Root{metadata.Root})
}
```

모니터링 및 진단

1. 성능 메트릭

데이터베이스 통계

```
func collectDBMetrics(nodeDB api.NodeDB) map[string]interface{} {
metrics := make(map[string]interface{})
// 크기 정보
if size, err := nodeDB.Size(); err == nil {
metrics["db_size_bytes"] = size
}
// 버전 정보
if latest, exists := nodeDB.GetLatestVersion(); exists {
```

```
metrics["latest_version"] = latest
}
metrics["earliest_version"] = nodeDB.GetEarliestVersion()
// 백엔드별 특화 메트릭
switch db := nodeDB.(type) {
case *badger.BadgerNodeDB:
metrics.update(collectBadgerMetrics(db))
case *pathbadger.BadgerNodeDB:
metrics.update(collectPathBadgerMetrics(db))
}
return metrics
}
```

BadgerDB 특화 메트릭

```
func collectBadgerMetrics(db *badger.DB) map[string]interface{} {
lsm, vlog := db.Size()
return map[string]interface{}{
"lsm_size": lsm,
"vlog_size": vlog,
"num_reads": db.Opts().NumGoroutines,
"num_writes": db.Opts().NumCompactors,
}
```

```
}
```

2. 건강성 검사

데이터 무결성 검사

```
func checkDatabaseIntegrity(nodeDB api.NodeDB) error {
latestVersion, exists := nodeDB.GetLatestVersion()
if !exists {
return fmt.Errorf("no finalized versions")
}
// 최신 버전의 모든 루트 검사
roots, err := nodeDB.GetRootsForVersion(latestVersion)
if err != nil {
return err
}
for _, root := range roots {
if !nodeDB.HasRoot(root) {
return fmt.Errorf("missing root: %v", root)
}
// 루트 노드 접근 테스트
if err := testRootAccess(nodeDB, root); err != nil {
return fmt.Errorf("root access failed: %w", err)
}
```

```
}
return nil
}
```

3. 문제 해결

일반적인 문제와 해결책

문제: 데이터베이스 크기 급증

```
// 해결책: 가비지 컬렉션 강제 실행
func forceGarbageCollection(nodeDB api.NodeDB) error {
// 오래된 버전 정리
latestVersion, _ := nodeDB.GetLatestVersion()
earliestVersion := nodeDB.GetEarliestVersion()
// 최근 10개 버전만 유지
keepVersions := uint64(10)
if latestVersion > earliestVersion+keepVersions {
pruneUntil := latestVersion - keepVersions
for v := earliestVersion; v < pruneUntil; v++ {
if err := nodeDB.Prune(v); err != nil {
return err
}
}
}
```

```
return nil
}
```

문제: 성능 저하

```
// 해결책: 캐시 크기 조정 및 압축
func optimizePerformance(cfg *api.Config) {
// 캐시 크기 증가
cfg.MaxCacheSize *= 2
// fsync 비활성화 (개발 환경에서만)
if isDevelopment() {
cfg.NoFsync = true
}
}
```

문제: 메모리 사용량 과다

```
// 해결책: 메모리 사용량 제한
func limitMemoryUsage(cfg *api.Config) {
// 시스템 메모리의 5%로 제한
totalMemory := getSystemMemory()
cfg.MaxCacheSize = totalMemory / 20
}
```

마이그레이션

백엔드 간 마이그레이션

```
func migrateBackend(oldDB, newDB api.NodeDB) error {
latestVersion, exists := oldDB.GetLatestVersion()
if !exists {
return fmt.Errorf("no data to migrate")
}
// 모든 버전을 순차적으로 마이그레이션
for version := oldDB.GetEarliestVersion(); version <= latestVersion; version+
roots, err := oldDB.GetRootsForVersion(version)
if err != nil {
return err
}
for _, root := range roots {
if err := migrateRoot(oldDB, newDB, root); err != nil {
return err
}
}
// 버전 완료 처리
if err := newDB.Finalize(roots); err != nil {
return err
```

```
}

}

return  nil

}
```

스토리지 백엔드 선택은 애플리케이션의 성능 요구사항, 데이터 크기, 운영 환경에 따라 결정되어야 합니다. BadgerDB는 일반적인 용도에 적합하며, PathBadger는 고성능이 필요한 환경에서 우수한 성능을 제공합니다.